RouteRec: Behavior-Guided Sparse Routing for Sequential Recommendation

Anonymous Author(s)

Abstract

Sequential recommendation models have become much better at using temporal order, user history, and side information to predict the next item. More recently, some studies have also introduced Mixture of Experts (MoE) into sequential recommendation and reported promising gains. This opens a natural next question: how can MoE be used more effectively in this setting?

We argue that the key issue is not only whether a model uses experts, but how it decides which experts should handle each input. In sequential recommendation, interaction logs show recurring behavioral patterns such as changes in pace, uncertainty in local transitions, repetition across nearby behavior, and differences in popularity exposure. These patterns suggest useful clues for routing, but they are not explicitly used in most MoE-based sequential recommenders.

RouteRec builds routing signals directly from ordinary interaction logs and uses them to guide expert selection. The model summarizes behavior at multiple temporal scopes, then applies a hierarchical sparse router so that different inputs can activate different expert subsets. Under a shared sessionized temporal protocol across six public datasets, RouteRec achieves the best overall seen-target ranking, with an Avg. Rank of 1.67 versus 3.56 for the next-best baseline. The clearest gains appear on KuaiRec, LastFM, and Foursquare. Further analysis shows a consistent pattern: RouteRec helps most when the behavioral structure that motivated its routing design is more strongly present in the data. Anonymous code: https://anonymous.4open.science/r/RouteRec/.

CCS Concepts

• **Information systems** → **Recommender systems**.

Keywords

sequential recommendation, sparse routing, conditional computation, behavioral cues, mixture of experts

ACM Reference Format:
Anonymous Author(s). 2026. RouteRec: Behavior-Guided Sparse Routing for Sequential Recommendation. In *Proceedings of the 20th ACM Conference on Recommender Systems (RecSys '26), September 28–October 2, 2026, Minneapolis, MN, USA.* ACM, New York, NY, USA, 19 pages.

1 Introduction

Sequential recommendation has advanced through stronger sequence models and richer use of behavioral context for next-item prediction [35]. These advances improve what the model represents. They still apply the same shared transformation to every prefix. This leaves an open question in sessionized sequential recommendation: should structurally different behavior always be processed by the same computation path?

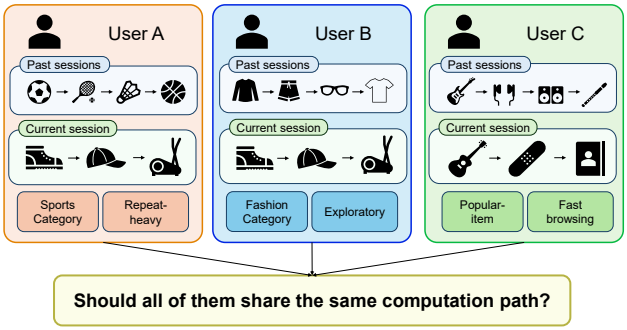


Figure 1: Sequential inputs can differ not only in item identity but also in pace, repetition, and local transition structure, even when they are processed by the same shared transformation.

Mixture of Experts (MoE) gives a direct way to address that question. It enables *conditional computation*: for each input, the model executes only a data-dependent subset of experts instead of the full shared capacity [11]. MoE has already been used in recommendation and sequential recommendation [21]. The router determines the resulting *computation path* by selecting which experts are activated.

Recent MoE work often advances in two directions. One direction improves routing mechanics, such as sparse activation efficiency and training stability [5]. The other direction improves the signal that defines expert allocation, so routing aligns with meaningful variation in the data [24]. In our setting, that variation comes from recurring differences in observed behavior that can justify different routes. We refer to the extent to which such variation creates a meaningful need for routed computation as *behavioral routing demand*. Once conditional computation is available, expert allocation is therefore no longer a fixed implementation detail. It becomes a modeling choice. This leads to our central question: in sessionized sequential recommendation, which recurring behavioral axes should define expert allocation?

Sessionized interactions suggest that the answer should come from the sequences themselves. They differ not only in which items appear, but also in how behavior unfolds over time. Some prefixes move at a steady pace, while others speed up or slow down sharply. Some prefixes branch into many plausible next steps, while others stay concentrated. Some show strong repetition or carryover from earlier sessions,

RecSys '26, Minneapolis, MN, USA
2026.

while others are shaped more by new exposure. A sessionized view makes these recurring patterns easier to inspect because it places short-term behavior and earlier user history in one frame.

We refer to the extent to which such recurring variation creates a meaningful need for routed computation as *behavioral routing demand*. In this paper, that demand appears repeatedly along four *recurring behavioral axes*: Tempo, which reflects pace and interval dynamics; Focus, which reflects concentration and switching in local transitions; Memory, which reflects repetition and carryover; and Exposure, which reflects popularity level and drift. These axes are motivation-level descriptors, not the router input itself. They tell us where routed computation is likely to matter. A related support condition is *context availability*, which asks whether reusable cross-session evidence is sufficiently available for broader-scope routing.

RouteRec operationalizes these axes as lightweight *behavioral cues* computed directly from ordinary interaction logs. The cues are read on a separate *routing control path*. The *sequential backbone* still handles next-item prediction. Routing is therefore not driven by prediction-optimized hidden states alone. RouteRec computes a cue vector at three routing stages: *macro scope* for broader history, *mid scope* for within-session evolution, and *micro scope* for short local transitions. These cue vectors are organized into four cue families: Tempo, Focus, Memory, and Exposure. A hierarchical sparse router then performs *hierarchical sparse expert allocation*, selecting a small, behavior-dependent expert subset at each stage.

Later experiments show a consistent pattern. RouteRec helps most when local branching is strong and repeated-session context is sufficiently available. The clearest gains appear on KuaiRec, LastFM, and Foursquare. The margin narrows when routing headroom is limited by weaker context availability or by strong shared-path suitability. We use this empirical pattern as supporting evidence for the routing-design view above, not as a post hoc scoring argument.

The main contributions of this work are:

- We formulate MoE routing in sessionized sequential recommendation as a routing-design question: expert allocation should be aligned with recurring behavioral axes in interaction logs, not driven by prediction-optimized hidden states alone.
- We propose **RouteRec**, which operationalizes this view through behavioral cue construction, coarse-to-fine multi-stage routing, and hierarchical sparse expert allocation on a routing control path separated from the sequential backbone.
- Under a shared sessionized temporal protocol across six public datasets, RouteRec achieves the best overall seen-target Avg. Rank, with the clearest gains on KuaiRec, LastFM, and Foursquare.

2 Related Work

We organize related work around the modeling question in this paper: what should define expert allocation in sessionized sequential recommendation? We first summarize mainstream sequential recommendation under shared computation. We then review how MoE is introduced in recommendation and sequential recommendation. Finally, we discuss recent MoE advances that clarify the distinction between routing mechanics and routing-signal design.

2.1 Sequential Recommendation and Shared-Path Baselines

Sequential recommendation predicts the next item from ordered user interactions [36]. Session-based recommendation is a closely related setting that partitions interactions into short sessions and predicts from current-session context [12, 35]. Early models established core sequence backbones, including factorized Markov transitions, convolutional architectures, and recurrent architectures [10, 28, 34]. Later models strengthened intent and transition modeling through attention and graph-based designs [19, 22, 37].

Transformer backbones such as SASRec and BERT4Rec became strong general references [15, 32]. Recent variants improve representation quality with contrastive objectives, frequency-aware bias, and state-space sequence modeling [4, 23, 26]. Across these families, most methods still apply one shared transformation path to all prefixes. This shared-path setup is exactly the context in which our routing question arises.

2.2 MoE in Recommendation and Sequential Recommendation

MoE introduces conditional computation through a router that activates only a subset of experts for each input [11, 14]. In recommendation, MMoE and PLE show that expert allocation can improve learning when different tasks require different specialization patterns [24, 33]. This established the practical value of expert allocation beyond simply enlarging model capacity.

MoE is now also used in sequential recommendation. M3SRec and HM4SR apply experts to multimodal and temporal structure in sequence modeling [1, 40]. FAME uses expert-style factorization to model fine-grained preference facets [21]. These results show that expert-based designs can help sequential recommendation. However, routing decisions are still often dominated by hidden representations or modality-specific states. This keeps open the design question we focus on: which recurring behavioral axes in sessionized logs should define expert allocation?

2.3 Recent MoE Advances for Routing Design

Recent MoE work often advances along two directions. One direction improves routing mechanics, including sparse activation, dispatch efficiency, and training stability [5, 30, 42, 43].

The other direction improves the signal that defines specialization, so expert allocation follows meaningful structure in data rather than only latent geometry [24, 33].

This distinction motivates RouteRec directly. In sessionized sequential recommendation, recurring differences in pace, transition ambiguity, memory pattern, and exposure pattern create *behavioral routing demand*. RouteRec follows the sparse-routing line for efficient conditional computation, but defines expert allocation through explicit *behavioral cues* aligned with these *recurring behavioral axes*. The cues are computed on a separate *routing control path*, while the *sequential backbone* remains focused on next-item prediction.

3 Problem and Design Challenges

3.1 Problem Definition

For each user, let $\mathcal{S}_u = [s_1, \ldots, s_M]$ denote the chronologically ordered sequence of sessions, where each session $s_m = [x_{m,1}, \ldots, x_{m,L_m}]$ contains items interacted with during one continuous activity window. The task is *next-item prediction*: given the current-session prefix $x_{m,1:t}$ and the user's past sessions $[s_1, \ldots, s_{m-1}]$, predict the next item $x_{m,t+1}$.

Beyond item identities, interaction logs commonly record when each interaction occurred and what broad category the item belongs to — temporal timestamps and coarse grouping labels such as category, genre, or artist. Corpus-level popularity statistics can also be derived from the training data. We assume these fields are available in the majority of standard recommendation benchmarks. The method is designed to work best when they are present and to degrade gracefully when some are missing.

3.2 Design Challenges

RouteRec starts from a simple premise: recurring behavioral variation in sessionized logs can create *behavioral routing demand*, that is, a meaningful need for routed computation. Three design challenges follow from this premise. They determine whether behavioral routing demand can be translated into stable and behavior-dependent routes, or whether routing collapses back into a near-shared transformation.

C1. Cue portability versus informativeness. Behavioral routing demand must be read from signals that are widely available across datasets. In practice, this means item identities, timestamps, and sometimes a coarse grouping field such as category or genre. The difficulty is that such evidence is narrow. A cue built only from these fields may be too weak to differentiate routes, yet a cue that depends on richer metadata is unavailable in many benchmark settings. The first challenge is therefore to construct *behavioral cues* that remain portable while still carrying enough behavioral meaning to support different computation paths.

C2. Temporal scope of behavioral evidence. Behavioral routing demand is not temporally uniform. Cross-session repetition and carryover become visible only over multiple past sessions. Within-session concentration builds gradually over the current prefix. Local transition pressure appears only in the most recent interactions before the target. If all of this evidence is merged into one undifferentiated routing signal, slow structural tendency and fast local dynamics become entangled. The second challenge is therefore to align routing decisions with the temporal scope at which each kind of behavioral evidence becomes reliable.

C3. Route commitment versus coverage. Even when routing reads meaningful cues at the right temporal scope, it can still fail to produce distinct routes. If weights spread broadly across experts, a repeat-heavy session and a fast exploratory session may activate almost the same mixture. The resulting model behaves little differently from a shared transformation. To avoid this collapse, routing must show *route commitment*: it must concentrate activation on a small, behavior-dependent expert subset. At the same time, it must preserve enough coverage to handle the full range of behavioral regimes seen in practice.

The next section addresses these challenges in the same order. It first identifies what observable evidence the router should read, then explains how that evidence should be separated by temporal scope, and finally shows how it is converted into committed sparse routes.

4 Proposed Method

RouteRec is a session-based sequential recommendation model that uses cue-driven sparse routing. It keeps two separate paths: a *sequential backbone* for next-item prediction, and a *routing control path* that reads behavioral cues from interaction sequences and selects experts. This separation keeps routing aligned with behavioral routing demand instead of hidden-state geometry alone.

4.1 RouteRec Design

Figure 2 summarizes RouteRec at the framework level. The design follows the three challenges in Section 3: what evidence routing should read, how that evidence should be separated over time, and how it should be turned into distinct routes.

C1. Behavioral Cue Construction. For C1, RouteRec fixes the routing evidence to simple metadata that are available before any model-specific transformation. These sources vary substantially across samples in ordinary sequential recommendation logs, and they directly reflect behavior that matters for routing.

- **Item identities**
- **Timestamps**
- **Optional coarse grouping labels**
- **Training-derived popularity statistics**

Section 4.2 turns these metadata into *behavioral cues*.

C2. Coarse-to-Fine Multi-Stage Routing. For C2, RouteRec separates routing evidence by temporal scope, because different behavioral evidence becomes reliable at different ranges.

- **Recent past sessions**
- **Current-session prefix**

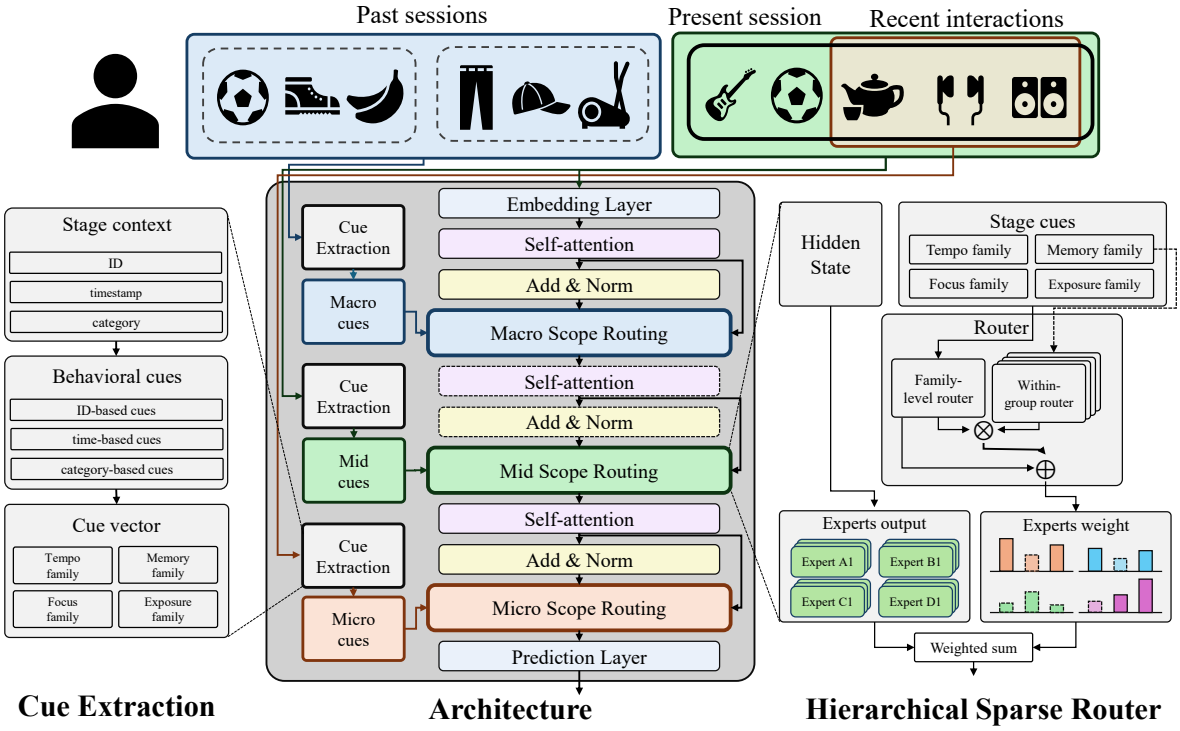


Figure 2: RouteRec framework. *Left — Cue Extraction*: interaction fields (ID, timestamp, category) are summarized into a cue vector organized by four cue families (Tempo, Focus, Memory, Exposure), computed separately at each temporal scope. *Center — Architecture*: sequential backbone with macro, mid, and micro routing stages in series. *Right — Hierarchical Sparse Router*: each stage uses the same two-level router — a family-level router selects which cue families drive computation, and a within-group router selects specific experts inside each chosen family. Details in Section 4.3 and Appendix A.

- **Most recent few interactions**

Section 4.3 maps these three windows to macro, mid, and micro routing stages.

C3. Hierarchical Sparse Expert Allocation. For C3, RouteRec organizes routing so cue information is used selectively rather than through one flat expert pool.

- **Expert groups aligned with cue families**
- **Within-group expert selection**
- **Sparse top-k activation**

Section 4.4 makes this selective use explicit.

Sections 4.2–4.4 now describe how these three choices are instantiated in the model. Extended layout variants are deferred to Appendix A.

4.2 Behavioral Cue Construction

The routing control path computes behavioral cues directly from interaction sequences, without passing through the sequential backbone. Given a window of interactions $\mathcal{R}$, a shared summary function $\mathcal{A}(\cdot)$ produces a fixed-size *cue vector* $\mathbf{f} = \mathcal{A}(\mathcal{R})$ from the selected sequence fields.

In the main design, $\mathcal{A}(\cdot)$ summarizes the selected metadata into four *cue families*. The Tempo family captures interaction pace and interval dynamics. The Focus family captures concentration and switching patterns over the coarsest grouping structure available. The Memory family captures repetition, recurrence, and carryover. The Exposure family captures popularity level and drift across the window.

This family structure keeps the routing input compact while preserving the main behavioral distinctions visible in ordinary sequential recommendation logs. Tempo and Memory are supported by item and time signals that are present in nearly all settings. Focus uses the coarsest grouping structure available, and falls back to a fixed missing-signal rule when that field is absent. Exposure uses popularity counts computed from the training corpus. Representative cue definitions are in Table 4; exact dimensions are in Appendix A.

The same four-family structure is applied at three temporal windows (macro, mid, micro), described in the next subsection. Because the cue template is fixed across all windows and datasets, no dataset-specific feature engineering is required.

4.3 Coarse-to-Fine Multi-Stage Routing

RouteRec separates routing by temporal scope and places one routing stage at each scope. For session s_m at prediction

target t, the three windows are:

$$\mathcal{R}_t^{\text{macro}} = [s_{\max(1,m-H)}, \ldots, s_{m-1}], \tag{1}$$

$$\mathcal{R}_t^{\text{mid}} = [x_{m,1}, \ldots, x_{m,t-1}], \tag{2}$$

$$\mathcal{R}_t^{\text{micro}} = [x_{m,\max(1,t-w)}, \ldots, x_{m,t-1}], \tag{3}$$

where H is the number of past sessions and w the local window size. Applying $\mathcal{A}(\cdot)$ to each window gives stage-specific cue vectors $\mathbf{f}_t^{(\text{macro})}$, $\mathbf{f}_t^{(\text{mid})}$, $\mathbf{f}_t^{(\text{micro})}$.

Macro reads slower cross-session tendency. Mid tracks how the current session develops. Micro reads the immediate transition context near the target.

The three stages are interleaved with self-attention blocks in the backbone. This keeps routing tied to the temporal role of each stage without expanding the architecture into many competing pathways. In the main configuration:

$$\underbrace{\text{self-attn}}_{\text{contextualize}} \rightarrow \text{macro} \rightarrow \text{mid} \rightarrow \underbrace{\text{self-attn}}_{\text{refresh}} \rightarrow \text{micro}.$$

Macro and mid are both *sequence-level* decisions. Cross-session tendency and within-session dynamics are properties of the whole session, not of individual positions — a session that has been repeat-heavy is repeat-heavy everywhere, not just at one token. It therefore makes sense to compute one routing decision per sequence (from averaged cue vectors) and apply it uniformly. Micro, by contrast, is *position-level*: local transition patterns differ at each step, so micro computes a separate routing decision per position. A second self-attention block before micro refreshes token interactions at the point where position-sensitive routing matters most. Full formulations are in Appendix A.

Section 4.4 describes how expert selection within each stage enforces route commitment.

4.4 Hierarchical Sparse Expert Allocation

RouteRec uses a two-level hierarchical router so cue families and expert specialization remain structurally separated. The router first decides which cue families matter, and then selects experts within the chosen families. This organization also supports *route commitment*: if routing spreads weight across all experts, behaviorally different sessions can still end up with nearly the same mixture.

RouteRec satisfies both requirements through a two-level hierarchical router. The E experts are grouped into G *expert groups* of C experts each ($E = GC$), where each group is aligned with one cue family (Tempo, Focus, Memory, or Exposure). Routing proceeds in two steps: first select which cue families drive computation (family-level), then select specific experts within each chosen family (within-group).

Router formulation. The router takes the cue vector $\mathbf{f}_t^{(s)}$ as input and produces two distributions via learned linear projections: a family-level distribution $p_{t,g}^{(s)}$ over expert groups, and a within-group distribution $r_{t,g,c}^{(s)}$ over experts inside each group. Their product $\pi_{t,g,c}^{(s)} = p_{t,g}^{(s)} \cdot r_{t,g,c}^{(s)}$ gives the joint weight for expert (g, c). The factored form matters because it lets the model use cue semantics explicitly: first decide which cue family is most relevant, then select the expert that should respond within that family. The same router structure is shared across all three stages; stages differ only in which cue vector they receive.

Sparse selection. Route commitment is enforced by hard top-k selection at both levels: only the top-k_g expert groups and the top-k_e experts within each are active, with their weights renormalized. All other experts contribute nothing to the output. Let $\text{TopK}(\mathbf{v}, k)$ be the index set of the k largest entries of $\mathbf{v}$; the sparse group weights $\bar{p}_{t,g}^{(s)}$ and within-group weights $\bar{r}_{t,g,c}^{(s)}$ are:

$$\bar{p}_{t,g}^{(s)} = \frac{p_{t,g}^{(s)}\, \mathbf{1}[g \in \text{TopK}(\mathbf{p}_t^{(s)}, k_g)]}{\sum_{g'} p_{t,g'}^{(s)}\, \mathbf{1}[g' \in \text{TopK}(\mathbf{p}_t^{(s)}, k_g)]}, \tag{4}$$

$$\bar{r}_{t,g,c}^{(s)} = \frac{r_{t,g,c}^{(s)}\, \mathbf{1}[c \in \text{TopK}(\mathbf{r}_{t,g,:}^{(s)}, k_e)]}{\sum_{c'} r_{t,g,c'}^{(s)}\, \mathbf{1}[c' \in \text{TopK}(\mathbf{r}_{t,g,:}^{(s)}, k_e)]}. \tag{5}$$

The stage output sums only over active experts ($\hat{\pi}_{t,g,c}^{(s)} = \bar{p}_{t,g}^{(s)}\, \bar{r}_{t,g,c}^{(s)}$):

$$\tilde{\mathbf{h}}_t^{(s)} = \sum_{g,c} \hat{\pi}_{t,g,c}^{(s)}\, \text{Expert}_{g,c}^{(s)}(\mathbf{h}_t^{\text{in},(s)}). \tag{6}$$

Sparsity here is about commitment, not computational efficiency: it ensures that two sessions with different cues cannot end up on the same route just because the soft weights happen to average out the same way. Dense soft mixing — where every expert contributes a small nonzero weight — is evaluated as an ablation in Appendix E. Configuration values (G, C, k_g, k_e) are in Appendix A.

4.5 Training Objective

RouteRec is trained end-to-end with one prediction loss and two routing regularizers. The routing regularizers are not optional add-ons: without them, cue-driven routing can degenerate in two distinct ways that the prediction loss alone cannot prevent.

Prediction loss. The main objective is standard next-item cross-entropy. Let $\mathbf{z}$ be the final logit vector over all items and y the target item:

$$\mathcal{L}_{\text{CE}} = -\log \frac{\exp(z_y)}{\sum_j \exp(z_j)}.$$

This loss drives the model to rank the correct next item highly, but it provides no direct signal about whether the routing is cue-sensitive or not.

Route consistency. The first failure mode: the router ignores behavioral cues and routes all sessions identically, effectively collapsing back to a shared feed-forward block. Route consistency pushes against this by requiring sessions with similar behavioral cues to produce similar routing distributions. For each stage s and sample i, let $\bar{\mathbf{f}}_i^{(s)} = \frac{1}{L_i} \sum_t \mathbf{f}_{i,t}^{(s)}$ be the session-level cue average and $\bar{\boldsymbol{\pi}}_i^{(s)} = \frac{1}{L_i} \sum_t \boldsymbol{\pi}_{i,t}^{(s)}$ the corresponding average routing distribution. We find each sample's

nearest cue neighbor in the mini-batch (pair set $\mathcal{P}_s$) and penalize the divergence between their routing distributions:

$$\mathcal{L}_{\text{cons}} = \frac{1}{|\mathcal{T}|} \sum_{s \in \mathcal{T}} \frac{1}{|\mathcal{P}_s|} \sum_{(i,j) \in \mathcal{P}_s} \text{JSD}\left(\bar{\boldsymbol{\pi}}_i^{(s)}, \bar{\boldsymbol{\pi}}_j^{(s)}\right).$$

This encourages the router to produce routing distributions that reflect the structure of the cue space, not just whatever pattern minimizes the prediction loss.

Routing logit stabilization. The second failure mode is subtler: even when the router tries to be cue-sensitive, the pre-softmax logits can grow very large during training. When this happens, the softmax saturates and routing collapses to a near-hard assignment, making small cue differences invisible to the router — the cue vector stops mattering in practice even though the router nominally reads it. We address this with z-loss [43], which penalizes the scale of the pre-softmax expert logits $a^s_{i,t,g,c}$ to keep the routing distribution in a well-behaved range where cue differences can produce meaningfully different outputs:

$$\mathcal{L}_z = \frac{1}{|\mathcal{T}|} \sum_{s \in \mathcal{T}} \frac{1}{N_s} \sum_{(i,t) \in \mathcal{V}_s} \left(\log \sum_{g,c} \exp a^s_{i,t,g,c} \right)^2,$$

where $\mathcal{V}_s$ is the set of valid positions and $N_s = |\mathcal{V}_s|$. The final objective is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}} + \lambda_z \mathcal{L}_z.$$

We do not include a load-balancing term. Classical MoE training often penalizes unequal expert usage to prevent collapse onto a small subset. Here the goal is the opposite: behaviorally distinct sessions should activate different, committed routes rather than distributing traffic evenly. Route consistency and logit stabilization together are sufficient for stable, cue-aligned specialization.

5 Experiments

5.1 Experimental Setup

We evaluate RouteRec on six public datasets spanning e-commerce, POI recommendation, short-video consumption, music listening, movie preference, and anonymous clickstream behavior: Beauty [25], Foursquare [39], KuaiRec [7], LastFM [3], ML-1M [9], and Retail Rocket [29]. Appendix B summarizes their statistics.

We report three ranking metrics: Hit Rate (HR@k), Normalized Discounted Cumulative Gain (NDCG@k), and Mean Reciprocal Rank (MRR@k) at $k \in \{5, 10, 20\}$. The primary comparison uses the seen-target view, where each candidate set consists of items observed during training. This is the standard choice for ID-based recommendation models, which cannot assign meaningful scores to items they have never seen during training [18]. Unless otherwise noted, all reported values are seen-target test scores on HR@10, NDCG@10, and MRR@20.

All datasets are preprocessed under the same sessionized temporal protocol. Rather than using fixed public splits, we build session-aware splits from scratch to ensure a consistent evaluation setting across all methods [12, 18, 27]. Interactions are ordered chronologically, sessionized with inactivity heuristics, and iteratively filtered to remove sessions shorter than 5 and items with fewer than 3 interactions [19, 37]. We use a 30-minute threshold for Foursquare, KuaiRec, LastFM, ML-1M, and Retail Rocket, and a 14-day threshold for the much sparser Beauty review stream [2, 16, 27]. KuaiRec and LastFM are large datasets; to keep computation tractable, we use a randomly sampled 20% subset of KuaiRec and a 3% subset of LastFM. All methods are evaluated on the same sampled data. After preprocessing, sessions are sorted by start time and split chronologically into 70%/15%/15% train/validation/test partitions. Appendix B gives the full preprocessing and held-out cleanup details.

We compare against GRU4Rec [10], SASRec [15], TiSASRec [20], DuoRec [26], FEARec [4], BSARec [31], FAME [21], DIF-SR [38], and FDSA [41]. All methods use the same processed files, the same optimization family, matched early stopping, and the same model-selection rule.

5.2 Main Comparison

Table 1 presents seen-target(observed in train set at least one time) ranking results (HR@10, NDCG@10, MRR@20) on all six datasets across six datasets using HR@10, NDCG@10, and MRR@20, with an Avg. Rank summary across all reported rows. RouteRec achieves the best overall Avg. Rank (1.67 vs. 3.56 for the next-best baseline FDSA), with the clearest margins on Foursquare, LastFM, and the sampled KuaiRec subset. On Beauty and Retail Rocket it remains close to the top, and on ML-1M it is competitive without leading.

The relative performance of the baselines is largely explained by the nature of our evaluation protocol, not by differences in tuning quality. Because sessions are short and prefixes are recency-sensitive, models that explicitly model temporal structure or recent transitions — such as SASRec and TiSASRec — are strong competitors throughout [15, 20]. BSARec, which performs well in longer-sequence settings, is less competitive here: its frequency-domain attention mechanism benefits from long, stable interaction histories, which sessionization breaks up [31]. DIF-SR, which relies on rich auxiliary side information, gains less because the auxiliary features available across our six datasets are intentionally kept lightweight and consistent, limiting what the side-information branch can add [38]. Since all methods share the same data, stopping rule, and candidate grid, these differences reflect the protocol and each method's inductive bias, not unequal tuning.

Seen-target gains are most consistent on Foursquare, LastFM, and KuaiRec (datasets with pronounced behavioral variation across sessions) and narrowest on ML-1M. This pattern is consistent with the core design premise: behavior-guided routing adds the most value when behavioral regimes shift materially across sessions.

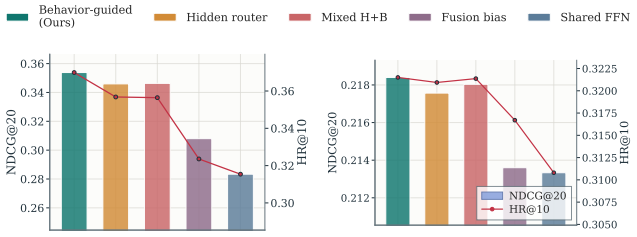
Table 1: Test results under the sessionized temporal protocol. Datasets marked with † are sampled processed subsets (KuaiRec 20%, LastFM 3%). For each method, we report the test metrics of the run achieving the highest mean test score over the nine reported metrics. Avg. Rank is computed over all rows in this table, where lower is better. The RouteRec column is shaded for readability.

Dataset	Metric	SASRec	GRU4Rec	TiSASRec	FEARec	DuoRec	BSARec	FAME	DIF-SR	FDSA	RouteRec
Beauty	HR@10	0.1324	0.0840	0.1235	0.1404	0.1347	0.0726	0.0497	0.1165	0.1318	**0.1561**
	NDCG@10	0.0800	0.0457	0.0831	0.0797	0.0793	0.0504	0.0303	0.0811	0.0918	**0.0948**
	MRR@20	0.0659	0.0361	0.0735	0.0634	0.0662	0.0450	0.0250	0.0716	**0.0822**	0.0794
Retail Rocket	HR@10	0.5321	0.5048	0.5131	0.5487	0.5478	0.4655	0.4612	0.4322	0.5343	**0.5608**
	NDCG@10	0.3907	0.3674	0.3755	0.3998	0.3994	0.3957	0.3943	0.3863	0.3848	**0.4148**
	MRR@20	0.3507	0.3290	0.3368	0.3581	0.3576	**0.3745**	0.3738	0.3709	0.3427	0.3737
Foursquare	HR@10	0.3049	0.2180	0.2976	0.3054	0.3195	0.2383	0.2417	0.3032	0.3042	**0.3215**
	NDCG@10	0.1929	0.1476	0.1926	0.1925	0.1970	0.1609	0.1604	0.2008	0.2014	**0.2042**
	MRR@20	0.1624	0.1286	0.1635	0.1613	0.1631	0.1393	0.1378	0.1723	**0.1726**	0.1715
ML-1M	HR@10	0.1423	0.1456	0.1448	0.1490	0.1555	0.1374	0.1533	0.0669	**0.1606**	0.1568
	NDCG@10	0.0717	0.0778	0.0756	0.0736	0.0810	0.0723	0.0818	0.0331	**0.0833**	0.0807
	MRR@20	0.0555	0.0621	0.0600	0.0559	0.0641	0.0570	0.0651	0.0256	**0.0655**	0.0631
LastFM†	HR@10	0.3775	0.3109	0.3715	0.3876	0.3850	0.3475	0.3359	0.3593	0.3762	**0.3965**
	NDCG@10	0.3097	0.2676	0.3029	0.3187	0.3066	0.3044	0.2972	0.3169	0.3247	**0.3307**
	MRR@20	0.2897	0.2558	0.2831	0.2981	0.2833	0.2915	0.2861	0.3051	0.3103	**0.3113**
KuaiRec†	HR@10	0.3284	0.3061	0.3216	0.3513	0.3468	0.3359	0.3307	0.2333	0.3453	**0.3669**
	NDCG@10	0.3177	0.2589	0.2889	0.3286	0.3215	0.3209	0.3188	0.2229	0.3197	**0.3467**
	MRR@20	0.3150	0.2462	0.2802	0.3227	0.3153	0.3170	0.3160	0.2201	0.3133	**0.3425**
Avg. Rank↓		**6.06**	**8.67**	**6.61**	**4.22**	**4.17**	**6.61**	**6.72**	**6.72**	**3.56**	**1.67**

5.3 Routing Control

A central design question is what the router should use as its input. Two options are available: hidden states, which summarize the sequence but are optimized for next-item prediction rather than behavioral character; and behavioral cues, computed directly from interaction logs on a separate control path. To test which input produces more consistent routing, we compare five variants: a shared FFN (no routing), hidden-only routing, mixed hidden+behavioral cue routing, additive fusion bias, and the final behavior-guided sparse router.

Figure 3 shows results on KuaiRec and Foursquare. Hidden-only routing improves over the shared FFN, and additive fusion bias adds some gain, but neither reaches the consistency of behavior-guided routing. Mixed routing, which combines hidden states and behavioral cues — performs well but remains below the fully separated design. The behavior-guided router, where behavioral cues drive expert selection through a control path that does not touch the sequential backbone, yields the most stable performance across both datasets. The practical takeaway is that the cleanest control boundary — not the most information-rich input — produces the most consistent routing.



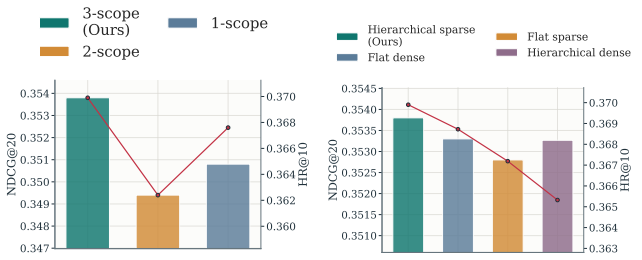
(a) KuaiRec **(b) Foursquare**

Figure 3: Routing-control ablations on KuaiRec and Foursquare. Five variants test what drives expert selection: a shared FFN baseline, hidden-state-only routing, mixed hidden+behavioral cue routing, additive fusion bias, and the final behavior-guided router.

5.4 Temporal Scope and Routing Design

RouteRec uses three routing stages (macro, mid, micro) and hierarchical sparse expert allocation. The three-stage structure raises the question of whether all scopes are necessary, and the hierarchical sparse design raises the question of whether the two-level selection adds anything over a flat top-k choice across all experts.

Figure 4 addresses both questions on KuaiRec. Panel (a) compares the three-stage design against the best-performing



(a) Temporal scopes (b) Routing organization

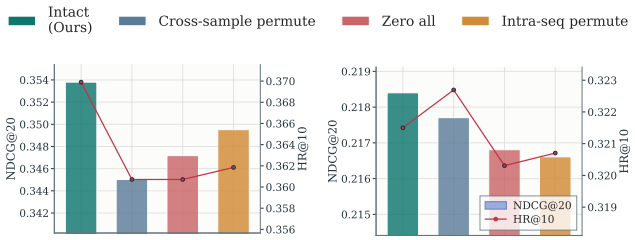
Figure 4: Design-justification ablations on KuaiRec. Panel (a) compares the full three-scope design against the best reduced two-scope and one-scope alternatives, including collapsed and parallel variants. Panel (b) compares hierarchical versus flat routing and sparse versus dense activation under comparable expert budgets.

two-scope and one-scope alternatives, where reduced variants include both scope-removal and parallel weighted-sum configurations. Removing any scope degrades performance, with the margin largest when the macro scope is absent: the macro scope captures cross-session behavioral tendency that neither the mid nor micro scope can recover. Panel (b) shows that flat routing (selecting top-k across all experts without group structure) and dense soft mixing both underperform the hierarchical sparse design under a comparable total expert budget. Route commitment — concentrating activation on a small, consistent expert subset — contributes beyond the expert count alone.

5.5 Cue Efficacy

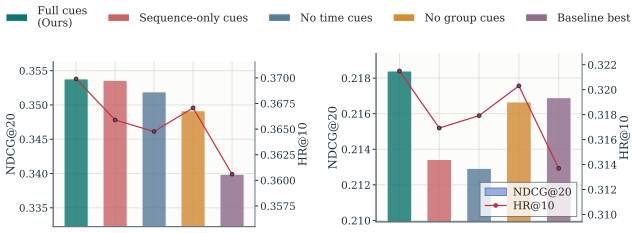
The routing-control and design ablations above establish that behavioral cues drive the routing decision better than hidden states do, and that the three-stage structure is necessary. A further question is whether the trained model actually depends on intact behavioral cues at inference time, or whether routing could be driven by an arbitrary cue assignment.

To test this, we hold the trained model fixed and apply three inference-time perturbations to the behavioral cues: cross-sample permutation (shuffling cue vectors across the batch), full zeroing (replacing all cues with zeros), and intra-sequence permutation (shuffling cue values within each sample). Figure 5 shows consistent performance drops across all three perturbations on both KuaiRec and Foursquare. Cross-sample permutation produces the largest drop, showing that the trained router reacts to the specific cue vector associated with each session rather than to an average routing pattern. Full zeroing and intra-sequence permutation also degrade performance, confirming that expert selection is sensitive to which behavioral cue is present and not merely to its scale. The result establishes that the MoE routing in RouteRec is functionally active: the behavioral cues directly determine which computation path is taken.



(a) KuaiRec (b) Foursquare

Figure 5: Inference-time cue perturbation on KuaiRec and Foursquare. The trained model is evaluated with intact behavioral cues, cross-sample permutation, full zeroing, and intra-sequence permutation. Performance drops consistently under all perturbations, confirming that expert selection depends on the identity of the cue vector.



(a) KuaiRec (b) Foursquare

Figure 6: Reduced-cue portability on KuaiRec and Foursquare. Each panel compares the full model against sequence-only cues, no-time-cue, and no-Focus-cue variants retrained from scratch, alongside the best baseline for reference.

5.6 Lightweight Cue Collection

Behavioral cues in RouteRec are derived from three metadata sources: item IDs (sequence-level statistics), timestamps (tempo and interval dynamics), and category labels (focus and concentration). A practical concern is whether the method requires all three sources, since timestamps and category labels are absent or unreliable in some deployment settings.

Figure 6 compares the full model against retrained variants that remove time cues, group (category) cues, or both, leaving only sequence-derived statistics. On KuaiRec, the sequence-only variant still exceeds the best baseline across both reported metrics, showing that item-ID-level cues alone are sufficient to maintain competitive routing. On Foursquare, performance with sequence-only cues remains close to the best baseline, with a small but stable gap relative to the full model. Removing only time cues or only group cues produces intermediate drops that differ between the two datasets, reflecting their different behavioral composition. The full model benefits from all three sources, but the method does not require rich metadata to remain useful: behavioral cues derived

from item sequences alone are enough to sustain meaningful routing across both datasets.

Full cutoff results and tuning details are reported in Appendices A–B.

6 Conclusion

RouteRec starts from a design question: in sessionized sequential recommendation with accessible cross-session history, can behavioral cues from ordinary interaction logs serve as an explicit control signal for conditional computation, rather than only as inputs for representation enrichment? The experiments support an affirmative reading from three angles. First, RouteRec improves next-item ranking most clearly on behaviorally dynamic datasets and remains competitive elsewhere, although the advantage narrows in more preference-stable settings such as ML-1M. Second, the gain is tied to behavioral cues, staged temporal decomposition, and sparse route commitment rather than indiscriminate expert insertion. Third, compact cues derived from ordinary interaction logs preserve a meaningful share of the benefit, keeping the method practical when side information is limited.

Taken together, the results suggest that behavior-guided routing can be both semantically meaningful and computationally selective. In the main configuration, route-guiding cues, coarse-to-fine routing, and hierarchical sparse expert allocation form a compact interface for conditional computation. Future work can extend that interface through adaptive sparsity budgets, transfer of routing priors across domains, more robust cue construction, and tighter deployment-aware constraints.

More broadly, the paper suggests a useful modeling stance for sessionized recommendation: not every gain from extra capacity should be interpreted as better sequence representation. Some of it can come from exposing a better control interface over when different computations should be used at all. RouteRec's contribution is to make that interface explicit, lightweight, and interpretable.

References

[1] Shuqing Bian, Xingyu Pan, Wayne Xin Zhao, Jinpeng Wang, Chao Wang, and Ji-Rong Wen. 2023. Multi-modal Mixture of Experts Representation Learning for Sequential Recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*. 110–119. doi:10.1145/3583780.3614978

[2] Simone Borg Bruun, Maria Maistro, and Christina Lioma. 2022. Learning Recommendations from User Actions in the Item-Poor Insurance Domain. In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys)*. Association for Computing Machinery, New York, NY, USA, 113–123. doi:10.1145/3523227.3546775

[3] Oscar Celma. 2010. *Music Recommendation and Discovery in the Long Tail*. Springer. doi:10.1007/978-3-642-13287-2

[4] Xinyu Du, Huanhuan Yuan, Pengpeng Zhao, Jianfeng Qu, Fuzhen Zhuang, Guanfeng Liu, Yanchi Liu, and Victor S. Sheng. 2023. Frequency Enhanced Hybrid Attention Network for Sequential Recommendation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 78–88. doi:10.1145/3539618.3591689

[5] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39. https://jmlr.org/papers/v23/21-0998.html

[6] B. G. Galuzzi, I. Giordani, A. Candelieri, R. Perego, and F. Archetti. 2020. Hyperparameter Optimization for Recommender Systems Through Bayesian Optimization. *Computational Management Science* 17, 4 (2020), 495–515. doi:10.1007/s10287-020-00376-3

[7] Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. 2022. KuaiRec: A Fully-Observed Dataset and Insights for Evaluating Recommender Systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management* (Atlanta, GA, USA) *(CIKM '22)*. 540–550. doi:10.1145/3511808.3557220

[8] Dmytro Gusak, Aleksandra Mojsilovic, Vassilis N. Ioannidis, and Christos Kozyrakis. 2025. Time to Split: Exploring Data Splitting Strategies for Offline Evaluation of Sequential Recommenders. arXiv:2507.16289 [cs.IR] doi:10.48550/arXiv.2507.16289

[9] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4 (2015), 1–19. doi:10.1145/2827872

[10] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1511.06939

[11] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive Mixtures of Local Experts. *Neural Computation* 3, 1 (1991), 79–87. doi:10.1162/neco.1991.3.1.79

[12] Dietmar Jannach, Massimo Quadrana, and Paolo Cremonesi. 2022. Session-Based Recommender Systems. In *Recommender Systems Handbook*. Springer. doi:10.1007/978-1-0716-2197-4_8

[13] Yuyi Ji, Aline B. de Oliveira, Raphael D. F. Mesquita, Diego H. C. G. Horta, and Leonardo B. Marinho. 2023. A Critical Study on Data Leakage in Recommender System Offline Evaluation. *ACM Transactions on Information Systems* 41, 3 (2023), 1–27. doi:10.1145/3569930

[14] Michael I. Jordan and Robert A. Jacobs. 1994. Hierarchical Mixtures of Experts and the EM Algorithm. *Neural Computation* 6, 2 (1994), 181–214. doi:10.1162/neco.1994.6.2.181

[15] Wang-Cheng Kang and Julian J. McAuley. 2018. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. doi:10.1109/ICDM.2018.00035

[16] Emanuel Lacic, Markus Reiter-Haas, Dominik Kowald, Manoj Reddy Dareddy, Junghoo Cho, and Elisabeth Lex. 2020. Using Autoencoders for Session-Based Job Recommendations. *User Modeling and User-Adapted Interaction* 30, 4 (2020), 617–658. doi:10.1007/s11257-020-09269-1

[17] Sara Latifi, Dietmar Jannach, and Andrés Ferraro. 2022. Sequential Recommendation: A Study on Transformers, Nearest Neighbors and Sampled Metrics. *Information Sciences* 609 (2022), 660–678. doi:10.1016/j.ins.2022.07.079

[18] Sahar Latifi, Nicola Mauro, and Dietmar Jannach. 2021. Session-Aware Recommendation: A Surprising Quest for the State-of-the-Art. *Information Sciences* 573 (2021), 354–375. doi:10.1016/j.ins.2021.05.048

[19] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. 2017. Neural Attentive Session-based Recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*. 1419–1428. doi:10.1145/3132847.3132926

[20] Jiacheng Li, Yujie Wang, and Julian J. McAuley. 2020. Time Interval Aware Self-Attention for Sequential Recommendation. In *Proceedings of the 13th ACM International Conference on Web Search and Data Mining (WSDM)*. 322–330. doi:10.1145/3336191.3371786

[21] Mingrui Liu, Sixiao Zhang, and Cheng Long. 2025. Facet-Aware Multi-Head Mixture-of-Experts Model for Sequential Recommendation. In *Proceedings of the 18th ACM International Conference on Web Search and Data Mining (WSDM)*. 127–135. doi:10.1145/3701551.3703552

[22] Qiao Liu, Yifu Zeng, Refuoe Mokhosi, and Haibin Zhang. 2018. STAMP: Short-Term Attention/Memory Priority Model for Session-Based Recommendation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1831–1839. doi:10.1145/3219819.3219950

[23] Ziwei Liu, Qidong Liu, Yejing Wang, Wanyu Wang, Pengyue Jia, Maolin Wang, Zitao Liu, Yi Chang, and Xiangyu Zhao. 2025. SIGMA: Selective Gated Mamba for Sequential Recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 12 (2025), 12264–12272. doi:10.1609/aaai.v39i12.33336

[24] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H. Chi. 2018. Modeling Task Relationships in Multi-Task Learning with Multi-Gate Mixture-of-Experts. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1930–1939. doi:10.1145/3219819.3220007

[25] Julian McAuley, Rahul Pandey, and Jure Leskovec. 2015. Inferring Networks of Substitutable and Complementary Products. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 785–794. doi:10.1145/2783258.2783381

[26] Ruihong Qiu, Zi Huang, Hongzhi Yin, and Zijian Wang. 2022. Contrastive Learning for Representation Degeneration Problem in Sequential Recommendation. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM)*. 813–823. doi:10.1145/3488560.3498433

[27] Massimo Quadrana, Alexandros Karatzoglou, Balázs Hidasi, and Paolo Cremonesi. 2017. Personalizing Session-based Recommendations with Hierarchical Recurrent Neural Networks. In *Proceedings of the Eleventh ACM Conference on Recommender Systems (RecSys)*. 130–137. doi:10.1145/3109859.3109896

[28] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing Personalized Markov Chains for Next-Basket Recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*. 811–820. doi:10.1145/1772690.1772773

[29] Retail Rocket. 2015. Retail Rocket Recommender System Dataset. https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset

[30] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=B1ckMDqlg

[31] Yehjin Shin, Jeongwhan Choi, Hyowon Wi, and Noseong Park. 2024. An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 8 (2024), 8984–8992. doi:10.1609/aaai.v38i8.28747

[32] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. 1441–1450. doi:10.1145/3357384.3357895

[33] Hongyan Tang, Junning Liu, Ming Zhao, Xudong Gong, Zhaohui Jin, Yu Lei, Min Zhang, and Xuan Li. 2020. Progressive Layered Extraction (PLE): A Novel Multi-Task Learning (MTL) Model for Personalized Recommendations. In *Proceedings of the 14th ACM Conference on Recommender Systems (RecSys)*. 269–278. doi:10.1145/3383313.3412236

[34] Jiaxi Tang and Ke Wang. 2018. Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding. In *Proceedings of the 11th ACM International Conference on Web Search and Data Mining (WSDM)*. 565–573. doi:10.1145/3159652.3159656

[35] Shoujin Wang, Longbing Cao, Yan Wang, Quan Z. Sheng, Mehmet A. Orgun, and Defu Lian. 2021. A Survey on Session-Based Recommender Systems. *Comput. Surveys* 54, 7 (2021), 1–38. doi:10.1145/3465401

[36] Shoujin Wang, Liang Hu, Yan Wang, Longbing Cao, Quan Z. Sheng, and Mehmet A. Orgun. 2019. Sequential Recommender Systems: Challenges, Progress and Prospects. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. 6332–6338. doi:10.24963/ijcai.2019/883

[37] Shu Wu, Yuyuan Tang, Yanqiao Zhu, Liang Wang, Xing Xie, and Tieniu Tan. 2019. Session-Based Recommendation with Graph Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 1 (2019), 346–353. doi:10.1609/aaai.v33i01.3301346

[38] Yueqi Xie, Peilin Zhou, and Sunghun Kim. 2022. Decoupled Side Information Fusion for Sequential Recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 1611–1621. doi:10.1145/3477495.3531963

[39] Dingqi Yang, Daqing Zhang, Vincent W. Zheng, and Zhiyong Yu. 2015. Modeling User Activity Preference by Leveraging User Spatial Temporal Characteristics in LBSNs. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 45, 1 (2015), 129–142. doi:10.1109/TSMC.2014.2327053

[40] Shengzhe Zhang, Liyi Chen, Dazhong Shen, Chao Wang, and Hui Xiong. 2025. Hierarchical Time-Aware Mixture of Experts for Multi-Modal Sequential Recommendation. In *Proceedings of the ACM Web Conference 2025 (WWW)*. 3672–3682. doi:10.1145/3696410.3714676

[41] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S. Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, and Xiaofang Zhou. 2019. Feature-level Deeper Self-Attention Network for Sequential Recommendation. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. 4320–4326. doi:10.24963/ijcai.2019/600

[42] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Y. Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. Mixture-of-Experts with Expert Choice Routing. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2202.09368

[43] Barret Zoph, Alexander V. Kolesnikov, Neal Wadhwa, Jonathan Huang, Andrew M. Dai, Noam Shazeer, Quoc V. Le, and Yonghui Wu. 2022. Designing Effective Sparse Expert Models. arXiv:2202.08906 [cs.LG] doi:10.48550/arXiv.2202.08906

A Exact RouteRec Instantiation and Sparse Routing Formulation

This appendix records the fixed paper instantiation and the representative cue bank used by RouteRec. Whereas the main text emphasizes the conceptual design axes, this section documents the implementation choices that are relevant for reproducibility and for interpreting the later ablations, with the sparse routing formulation made explicit.

A.1 Main Paper Configuration

Table 2: Fixed main configuration used in the paper.

Aspect	Main paper configuration
Backbone layout	self-attn → macro block → mid block → self-attn → micro block
Stage execution	Serial
Routing granularity	Macro and mid: sequence-wise; micro: position-wise
Router composition	Shared hierarchical sparse router at all three stages
Expert activation	Top-3 expert groups out of 4; top-2 experts per selected group
Active experts	6 out of 12 total per routed decision
History windows	$H = 5$ sessions (macro); $w = 5$ positions (micro)
Core auxiliary terms	Route consistency + z-loss; no load balancing

The placement of the two attention layers reflects the intended temporal roles of the stages. The first attention layer contextualizes the raw prefix before any routing is applied. Macro and mid then operate consecutively because both provide slower sequence-level control over the same contextualized sequence state: macro captures broader recent-history tendency, while mid refines within-session evolution. A second attention layer is inserted only before micro so that the final local router receives a refreshed token interaction state immediately before prediction. This layout was the most stable high-performing architecture in our architecture sweep.

This table serves as a reproducibility summary rather than as a search-space description. Architectural choices are fixed across the paper track, whereas capacity and optimization values are selected per dataset from a narrow candidate set and are therefore reported as compact ranges only when no single global value is shared across all reported runs.

A.2 Implementation Specification and Minimal Execution Notes

Table 3: Implementation-level settings used in the selected paper track.

Item	Reported setting
Cue families	$G = 4$ expert groups: Tempo, Focus, Memory, Exposure
Experts per group	$C \in \{2, 3, 4\}$; total experts $E = 4C$
Main sparse setting	$C = 3$, top-3 expert groups, top-2 experts per selected group
Cue dimensionality	16 scalars per stage
Router hidden width	Representative range 48–128
Routing temperature	Macro 1.0; mid and micro 1.2
Consistency neighbors	$k_{\mathrm{nn}} = 1$ nearest non-self sample per stage
Missing-signal handling	Broadcast when available; zero-fill focus cues if grouping field absent

Minimal execution notes. The implementation uses three simple rules that are easy to miss if one only reads the high-level method description. First, macro and mid cues are computed once per example and then broadcast over the routed positions, whereas micro cues are evaluated at the position level. Second, top-k masking is followed by renormalization inside the active support rather than by leaving the logits in an unnormalized sparse state. Third, if a dataset does not expose the grouping field needed for focus-family cues, the implementation falls back to the fixed missing-signal policy in Table 3 rather than silently redefining the cue family per dataset.

A.3 Representative Behavioral Cue Families

This cue bank is intentionally compact. The feature families are hand-designed in the sense that they encode behavioral hypotheses, but the design goal is to keep them as a small set of summaries that can serve as a stable routing interface across datasets. In practice, the macro bank is instantiated from the recent-session window used by the selected paper instantiation, while mid and micro are computed from the current session prefix at different temporal scopes. The main text therefore reports the family structure and representative examples, while this appendix provides the implementation-oriented view.

B Dataset, Evaluation, and Full Ranking Results

All experiments were conducted on machines equipped with NVIDIA RTX 3090 24 GB GPUs. Each training or evaluation process used a single GPU; multiple GPUs were used only to run independent experiments in parallel, not for multi-GPU training within a single run. This appendix summarizes the six datasets and makes the evaluation protocol explicit, since recent work has shown that session construction and split strategy can materially affect offline recommender results [8, 13]. The original releases are cited here for completeness [3, 7, 9, 25, 29, 39]. The dataset descriptions emphasize domain differences and signal availability because the routing branch depends on which behavioral evidence is exposed in each corpus.

Table 4: Representative cue summaries used by RouteRec in the main configuration. Each stage uses four semantic families with four scalar summaries per family; the table lists representative examples rather than every implementation detail.

Family	Macro (recent-session tendency)	Mid (within-session evolution)	Micro (short-horizon pressure)
Tempo	Context-valid ratio, last inter-session gap, mean pace, pace trend	Valid-prefix ratio, mean/std interaction interval, session age	Last gap, short-window mean gap, short-window std, deviation from mid-scale pace
Focus	Theme entropy, dominant-theme mass, theme repeat rate, theme shift rate	Category entropy, category top-1 mass, category switch rate, category uniqueness	Immediate category switch, mismatch to recent category history, suffix entropy, suffix uniqueness
Memory	Repeat intensity, adjacent category overlap, adjacent item overlap, repeat trend	Item uniqueness, repeat rate, novelty rate, longest item run	Reconsumption flag, suffix reconsumption rate, suffix item uniqueness, suffix longest run
Exposure	Mean popularity, popularity spread, popularity entropy, popularity trend	Mean popularity, popularity std, popularity entropy, popularity drift	Last-item popularity, suffix popularity spread, suffix popularity entropy, deviation from mid-scale popularity

B.1 Datasets and Protocol

Table 5: Datasets used in the experiments. Datasets marked with † are sampled processed subsets.

Dataset	Domain	Interactions	Sessions	Items	Avg. len.
Beauty	E-com.	33,488	4,243	3,625	7.9
Foursquare	POI	145,238	25,369	30,588	5.7
KuaiRec†	Video	287,411	24,458	6,477	11.8
LastFM†	Music	470,408	25,089	52,510	18.8
ML-1M	Movie	575,281	14,539	3,533	39.6
Retail Rocket	Browse	821,243	153,092	90,211	5.4

Table 5 reports the statistics of the final processed datasets used in the experiments rather than the raw corpus sizes. In particular, KuaiRec corresponds to the 20% processed subset used in our runs, and LastFM corresponds to the 3% processed subset.

For reproducibility, the protocol is summarized as follows.

(1) Sort all interactions chronologically within each dataset.
(2) Reconstruct sessions with a dataset-specific inactivity threshold.
(3) Iteratively prune sessions with length below 5 and items with support below 3 until the constraints are simultaneously satisfied.
(4) Sort the resulting sessions by session start time.
(5) Assign the earliest 70% of sessions to train and treat the latest 30% as a held-out tail.
(6) Build validation and test from the held-out tail under chronology-preserving cleanup.
(7) Report seen-target metrics as the primary view and keep unseen-target behavior as supplementary accounting.

Beauty and Retail Rocket stress sparse e-commerce revisit behavior, Foursquare and KuaiRec emphasize stronger short-horizon intent changes, and LastFM and ML-1M provide longer preference-driven histories. This spread makes it possible to examine whether behavior-guided routing is primarily beneficial in highly dynamic settings or remains competitive when preference structure is more stable.

Signal availability also differs in a way that matters for routing. Beauty exposes item, time, and product-category cues after low-rating removal. Foursquare includes item, timestamp, and venue-category structure with bursty check-in dynamics. KuaiRec offers item, time, and coarse content-side metadata in short-video sessions with strong short-term intent shifts. LastFM provides repeated listening histories with artist-side signals, while ML-1M contributes longer preference-driven sequences with genre information. Retail Rocket is the sparsest side-information setting, relying mainly on anonymous event streams with item and time signals.

We position our setup as a sessionized sequential evaluation protocol assembled from established components, not as the canonical raw-sequence protocol for sequential recommendation [12, 18, 27]. Sessions are reconstructed post hoc with inactivity heuristics, which is standard practice when logs do not provide explicit session boundaries. The thresholds are adapted to the logging regime of each corpus: Foursquare, KuaiRec, LastFM, ML-1M, and Retail Rocket use a 30-minute inactivity threshold, consistent with dense-log session-aware practice [16, 27], whereas Beauty uses a 14-day threshold because review timestamps are substantially more diffuse. More generally, this threshold choice reflects the observation that sparse and item-poor domains often require longer or more domain-specific notions of a session than dense clickstream data [2]. After sessionization, we repeatedly remove sessions shorter than 5 and items with fewer than 3 interactions until no further pruning is required. This iterative filtering follows the same general rationale as common session-based benchmark preprocessing, where short sessions and very low-support items are filtered to stabilize next-item supervision and evaluation [19, 37].

Session splits are constructed at the session level after sorting by session start time. The earliest 70% of sessions are assigned to training, and the remaining 30% are treated as a held-out tail. This tail is first divided into validation and test using stratification over three factors: whether the target item already appears in the prefix, the target item's popularity bucket with respect to the training split, and the session-length bucket. For the final evaluation files, we then merge the held-out sessions, restore chronological order, and re-split them contiguously into equal validation and test halves. This chronology-preserving design is meant to reduce

leakage and keep the offline protocol aligned with temporal deployment order [8, 13].

For the primary evaluation view, we report seen-target metrics only. Concretely, in the held-out files, non-target rows whose items are unseen in training are removed, and unseen targets are retained only for supplementary accounting rather than the main comparison. This matches long-standing session-based and session-aware evaluation practice for ID-based recommendation models, which cannot rank targets outside the training vocabulary and therefore typically separate unseen-target behavior from the primary benchmark table [18]. We report HR, NDCG, and MRR at $k \in \{5, 10, 20\}$ for all methods.

For the result tables, each method is represented by the run with the highest mean seen-target test score over the nine reported metrics under the same selection rule used throughout the paper.

All methods use the same processed train/validation/test partitions. RouteRec's additional macro, mid, and micro cues are deterministic summaries derived from those same observed histories, so the comparison should be read as evaluating different ways of exploiting the shared sessionized interaction context rather than different access to held-out information.

B.2 Model Selection and Tuning Protocol

Because this benchmark is built from a sessionized temporal protocol rather than a canonical raw-sequence leaderboard, the model-selection procedure is stated explicitly. Prior benchmark studies have shown that both baseline quality and model ranking can change materially once methods are re-evaluated under a common framework and retuned under a shared protocol [17, 18]. Accordingly, the comparison does not rely on paper defaults alone.

Our reporting protocol follows three simple rules. First, every model is tuned within a predefined bounded search space that is fixed before the final comparison: learning rate is sampled log-uniformly within a declared dataset-specific interval, while the remaining hyperparameters are drawn from small predeclared candidate sets. Second, these bounded spaces cover a shared core of optimization hyperparameters together with a small number of model-specific controls, so that the search is broad enough to avoid trivial under-tuning but still budgeted tightly enough to remain comparable across methods. Third, the final reported result for every model is the test performance of the configuration that maximizes the mean validation score over the nine reported metrics. When adaptive search is used for computational efficiency, it is restricted to the same predeclared bounds and candidate sets and should therefore be read as an efficient implementation of a controlled bounded search rather than as open-ended retuning [6].

The final paper track therefore uses bounded search spaces rather than unconstrained retuning. In the paper-facing comparison, baselines and RouteRec are aligned to the same shared optimization pool, and model families differ only through a small number of architecture-specific additions. For RouteRec, the selected three-stage routing layout remains fixed, but depth, hidden dropout, and routing-side controls are still tuned within bounded candidate sets. To keep the appendix concise but still reproducible, Table 6 separates the shared optimization pool from the baseline-family and RouteRec-specific additions, and lists the dataset-specific learning-rate intervals explicitly.

B.3 Full Ranking Results

Table 7 restores the full cutoff grid summarized more compactly in Table 1. This presentation preserves the same selection rule as the main text while exposing the complete $k \in \{5, 10, 20\}$ breakdown.

The appendix table confirms that the same qualitative ordering persists across cutoffs: RouteRec is strongest overall, SASRec and TiSASRec remain robust reference points, and the relatively weaker BSARec and DIF-SR results are consistent with the sessionized short-prefix regime discussed in Section 5.2 rather than with a change in selection or evaluation policy.

This table makes it possible to verify whether the observed trends persist beyond the compact main-text summary under the same selection rule.

Table 6: Bounded candidate grids used in the final paper track.

(a) Shared Optimization Pool

Parameter	Candidate sets
Learning rate	Dataset-specific log-uniform interval; see panel (d)
Weight decay	$\{5 \times 10^{-7}, 10^{-6}, 10^{-5}, 5 \times 10^{-5}, 10^{-4}, 1.5 \times 10^{-4}\}$
Max history length	$\{10, 20, 30\}$ or $\{10, 20, 30, 50\}$
Width / embedding	$\{64, 96, 112, 128, 160\}$ or $\{96, 112, 128, 160, 192\}$
Inner width	$\{128, 192, 224, 256, 320\}$ or $\{192, 224, 256, 320, 384\}$
Layers / heads	Layers $\{1, 2, 3, 4\}$; heads $\{1, 2, 4, 8\}$ if applicable
Dropout	Hidden / ratio: $\{0.10, 0.12, 0.13, 0.15, 0.18\}$ Recurrent: $\{0.10, 0.15, 0.20, 0.25, 0.30\}$ Attention: $\{0.06, 0.08, 0.10, 0.12, 0.15\}$ or $\{0.08, 0.10, 0.12, 0.15, 0.20\}$

(b) Baseline-Family Additions

Model(s)	Additional candidate sets
TiSASRec	Time span $\{64, 128, 256, 384, 512\}$
GRU4Rec	Recurrent dropout $\{0.10, 0.15, 0.20, 0.25, 0.30\}$
DuoRec / FEARec	$\tau\{0.16, 0.18, 0.20, 0.22, 0.24\}$ Contrastive weight $\{0.02, 0.03, 0.04, 0.05, 0.06\}$ Semantic weight $\{0, 0.04, 0.05, 0.08, 0.10\}$ or $\{0.04, 0.05, 0.08, 0.10, 0.12\}$
BSARec	$\alpha\{0.35, 0.50, 0.55, 0.70\}$ $c\{2, 3, 5, 7\}$
DIF-SR / FDSA	Attribute width $\{96, 128, 160, 192\}$ λ_{attr} $\{0.08, 0.09, 0.10, 0.12, 0.14\}$ or $\{0.09, 0.10, 0.12, 0.14, 0.15\}$
DIF-SR	Fusion $\{\text{gate}, \text{sum}, \text{concat}\}$
FAME	Expert count $\{2, 3, 4, 5, 6\}$

(c) RouteRec-Specific Additions

Parameter	Candidate sets
Backbone depth / dropout	Depth: $\{1, 2, 3\}$ Dropout: $\{0.10, 0.12, 0.14, 0.15, 0.16, 0.17\}$ $\{0.18, 0.19, 0.20, 0.22, 0.24\}$
Weight decay	$\{3.125, 5, 6, 10, 15, 20, 50\} \times 10^{-7}$
Routing capacity	Expert scale $\{2, 3, 4\}$ or $\{2, 3, 4, 5\}$ for KuaiRec router width $\{32, 64, 96, 128\}$
Routing dropout	Feature $\{0, 0.03, 0.05, 0.10\}$ attention $\{0.05, 0.06, 0.08, 0.10, 0.12\}$
Regularization	$\lambda_{\text{cons}}\{0, 2.5, 5, 8, 12\} \times 10^{-4}$ $\lambda_z\{0, 5, 10, 20\} \times 10^{-5}$
Cue bank	Cue bank size $\{10, 12, 16, 24\}$ router inner width per cue family (small preset)

(d) Dataset-Specific Learning-Rate Intervals

Dataset	Unified log-uniform LR interval
Beauty	1.5e-4 ∼ 2e-3
Foursquare	1.5e-4 ∼ 2.2e-3
KuaiRec	3e-4 ∼ 5e-3
LastFM	8e-5 ∼ 1.2e-3
ML-1M	1.5e-4 ∼ 2.2e-3
Retail Rocket	1.5e-4 ∼ 2.2e-3

Table 7: Full seen-target experimental results across all cutoffs. Datasets marked with † are sampled processed subsets. For each method, we report the seen-target test metrics of the run achieving the highest mean seen-target test score over the nine reported metrics. Avg. Rank is computed over all rows in this table, where lower is better. The RouteRec column is shaded for readability.

Dataset	Metric	SASRec	GRU4Rec	TiSASRec	FEARec	DuoRec	BSARec	FAME	DIF-SR	FDSA	RouteRec
Beauty	HR@5	0.1038	0.0573	0.0918	0.1003	0.0993	0.0630	0.0372	0.0993	0.1041	**0.1087**
	HR@10	0.1324	0.0840	0.1235	0.1404	0.1347	0.0726	0.0497	0.1165	0.1318	**0.1561**
	HR@20	0.1637	0.1146	0.1630	0.1757	0.1958	0.0955	0.0592	0.1385	0.1681	**0.2009**
	NDCG@5	0.0707	0.0372	0.0729	0.0669	0.0678	0.0474	0.0263	0.0756	**0.0830**	0.0798
	NDCG@10	0.0800	0.0457	0.0831	0.0797	0.0793	0.0504	0.0303	0.0811	0.0918	**0.0948**
	NDCG@20	0.0879	0.0533	0.0931	0.0889	0.0948	0.0563	0.0327	0.0867	0.1011	**0.1062**
	MRR@5	0.0598	0.0307	0.0666	0.0556	0.0571	0.0422	0.0228	0.0677	**0.0761**	0.0701
	MRR@10	0.0637	0.0341	0.0707	0.0608	0.0619	0.0434	0.0244	0.0700	**0.0796**	0.0762
	MRR@20	0.0659	0.0361	0.0735	0.0634	0.0662	0.0450	0.0250	0.0716	**0.0822**	0.0794
Retail Rocket	HR@5	0.4610	0.4312	0.4433	0.4730	0.4711	0.4411	0.4398	0.4262	0.4579	**0.4862**
	HR@10	0.5321	0.5048	0.5131	0.5487	0.5478	0.4655	0.4612	0.4322	0.5343	**0.5608**
	HR@20	0.5988	0.5719	0.5810	0.6234	0.6194	0.4887	0.4812	0.4365	0.6053	**0.6315**
	NDCG@5	0.3676	0.3436	0.3528	0.3752	0.3745	0.3877	0.3874	0.3844	0.3600	**0.3907**
	NDCG@10	0.3907	0.3674	0.3755	0.3998	0.3994	0.3957	0.3943	0.3863	0.3848	**0.4148**
	NDCG@20	0.4076	0.3844	0.3926	0.4187	0.4175	0.4015	0.3994	0.3874	0.4028	**0.4327**
	MRR@5	0.3365	0.3145	0.3227	0.3427	0.3423	0.3696	0.3695	**0.3698**	0.3274	0.3588
	MRR@10	0.3461	0.3243	0.3321	0.3529	0.3526	**0.3729**	0.3724	0.3706	0.3377	0.3688
	MRR@20	0.3507	0.3290	0.3368	0.3581	0.3576	**0.3745**	0.3738	0.3709	0.3427	0.3737
Foursquare	HR@5	0.2350	0.1741	0.2388	0.2403	0.2438	0.1995	0.1954	0.2458	0.2514	**0.2559**
	HR@10	0.3049	0.2180	0.2976	0.3054	0.3195	0.2383	0.2417	0.3032	0.3042	**0.3215**
	HR@20	0.3635	0.2593	0.3501	0.3620	0.3760	0.2773	0.2800	0.3536	0.3524	**0.3776**
	NDCG@5	0.1703	0.1334	0.1735	0.1714	0.1726	0.1483	0.1455	0.1821	**0.1843**	0.1829
	NDCG@10	0.1929	0.1476	0.1926	0.1925	0.1970	0.1609	0.1604	0.2008	0.2014	**0.2042**
	NDCG@20	0.2078	0.1580	0.2059	0.2068	0.2114	0.1708	0.1701	0.2135	0.2136	**0.2184**
	MRR@5	0.1489	0.1199	0.1519	0.1486	0.1491	0.1313	0.1289	0.1611	**0.1621**	0.1588
	MRR@10	0.1583	0.1258	0.1599	0.1574	0.1591	0.1366	0.1351	0.1688	**0.1692**	0.1676
	MRR@20	0.1624	0.1286	0.1635	0.1613	0.1631	0.1393	0.1378	0.1723	**0.1726**	0.1715
ML-1M	HR@5	0.0858	0.0984	0.0906	0.0906	0.0977	0.0892	0.0968	0.0384	**0.1022**	0.1005
	HR@10	0.1423	0.1456	0.1448	0.1490	0.1555	0.1374	0.1533	0.0669	**0.1606**	0.1568
	HR@20	0.2157	0.2181	0.2220	0.2234	0.2402	0.2018	0.2243	0.1052	**0.2426**	0.2353
	NDCG@5	0.0536	0.0626	0.0582	0.0549	0.0624	0.0568	0.0637	0.0240	**0.0645**	0.0627
	NDCG@10	0.0717	0.0778	0.0756	0.0736	0.0810	0.0723	0.0818	0.0331	**0.0833**	0.0807
	NDCG@20	0.0902	0.0960	0.0950	0.0923	0.1022	0.0884	0.0996	0.0427	**0.1038**	0.1004
	MRR@5	0.0431	0.0510	0.0476	0.0432	0.0508	0.0463	**0.0529**	0.0192	0.0522	0.0504
	MRR@10	0.0505	0.0572	0.0548	0.0508	0.0584	0.0527	**0.0603**	0.0230	0.0600	0.0577
	MRR@20	0.0555	0.0621	0.0600	0.0559	0.0641	0.0570	0.0651	0.0256	**0.0655**	0.0631
LastFM†	HR@5	0.3471	0.2836	0.3405	0.3630	0.3521	0.3312	0.3173	0.3389	0.3494	**0.3693**
	HR@10	0.3775	0.3109	0.3715	0.3876	0.3850	0.3475	0.3359	0.3593	0.3762	**0.3965**
	HR@20	0.3999	0.3338	0.3997	0.4103	0.4101	0.3615	0.3554	0.3825	0.4020	**0.4194**
	NDCG@5	0.2999	0.2588	0.2929	0.3108	0.2959	0.2992	0.2911	0.3103	0.3161	**0.3218**
	NDCG@10	0.3097	0.2676	0.3029	0.3187	0.3066	0.3044	0.2972	0.3169	0.3247	**0.3307**
	NDCG@20	0.3154	0.2735	0.3101	0.3245	0.3129	0.3080	0.3020	0.3228	0.3313	**0.3365**
	MRR@5	0.2841	0.2506	0.2770	0.2933	0.2771	0.2884	0.2822	0.3008	0.3050	**0.3059**
	MRR@10	0.2881	0.2542	0.2812	0.2965	0.2815	0.2906	0.2847	0.3035	0.3085	**0.3096**
	MRR@20	0.2897	0.2558	0.2831	0.2981	0.2833	0.2915	0.2861	0.3051	0.3103	**0.3113**
KuaiRec†	HR@5	0.3195	0.2770	0.3004	0.3397	0.3270	0.3281	0.3255	0.2275	0.3270	**0.3494**
	HR@10	0.3284	0.3061	0.3216	0.3513	0.3468	0.3359	0.3307	0.2333	0.3453	**0.3669**
	HR@20	0.3379	0.3367	0.3429	0.3688	0.3677	0.3464	0.3430	0.2411	0.3654	**0.3949**
	NDCG@5	0.3148	0.2493	0.2820	0.3249	0.3152	0.3184	0.3172	0.2210	0.3140	**0.3410**
	NDCG@10	0.3177	0.2589	0.2889	0.3286	0.3215	0.3209	0.3188	0.2229	0.3197	**0.3467**
	NDCG@20	0.3200	0.2666	0.2942	0.3330	0.3269	0.3236	0.3220	0.2248	0.3248	**0.3538**
	MRR@5	0.3133	0.2400	0.2759	0.3200	0.3112	0.3152	0.3144	0.2188	0.3097	**0.3382**
	MRR@10	0.3144	0.2441	0.2788	0.3215	0.3138	0.3162	0.3151	0.2196	0.3120	**0.3406**
	MRR@20	0.3150	0.2462	0.2802	0.3227	0.3153	0.3170	0.3160	0.2201	0.3133	**0.3425**
Avg. Rank↓		**6.07**	**8.63**	**6.56**	**4.41**	**4.41**	**6.52**	**6.72**	**6.55**	**3.40**	**1.74**

C Where RouteRec Helps Most: Special Bins

This section examines coarse difficulty-oriented bins such as session length, target frequency, and optional user history, with the goal of showing where the average RouteRec gain is concentrated. The reported analyses distribute gains across session-length, item-frequency, and optional user-history regimes rather than treating the average result as a black box.

Table 8: Special-bin definitions and sample counts used in the appendix gain analysis.

Bin family	Definition rule	Visualization
Session length	Short / medium / long session buckets from the observed prefix length	Figure 7(a)
Target frequency	Rare / mid / frequent target buckets under the train split	Figure 7(b)
User history	Optional low-history / medium-history / heavy-history grouping	Supporting figure

RouteRec produces its clearest seen-target margin in behaviorally heterogeneous or otherwise difficult regimes, consistent with the pattern reported in the main comparison (Section 5.2).

D Extended Framework and Structural Ablations

This section extends the design-justification study in Section 5.4 by examining broader temporal-layout and cue-organization variants. Figure 8 collects cue-family organization controls and temporal-layout variants that are too detailed for the main text. The goal is to show that the selected architecture is a stable design choice rather than an arbitrary one.

Panel (a) tests whether organizing experts into cue-family-aligned groups matters beyond a flat scalar bag. Panel (b) checks whether the macro, mid, and micro roles correspond to meaningfully distinct temporal views rather than just repeated copies of the same router, and whether the selected stage ordering remains the most stable high-performing layout among the temporal controls.

E Sparse Routing Design Variants

Table 9 tests whether the selected sparse hierarchy is an arbitrary engineering choice or a meaningful part of the RouteRec design. This section directly supports the design-justification study in Section 5.4 by contrasting dense (soft) mixing, flat sparse controls, and hierarchical top-k variants under comparable activation budgets.

Table 9: Sparse routing design variants, ranging from a dense reference to progressively sparser hierarchical configurations under comparable total expert pools.

Variant	MRR@20	Active experts	Observation
Dense full mixture	0.336	12	Reference control
Flat sparse top-6	0.335	6	Same active count without hierarchy
Top-2 families, top-1ex	0.332	2	Extremely sparse reference
Top-2 families, top-2ex	0.335	4	Strong family commitment, tight capacity
Top-3 families, top-2ex	**0.339**	6	**Selected main setting**
Top-4 families, top-2ex	0.337	8	Weaker family commitment
Top-3 families, top-3ex	0.338	9	Larger active-compute budget reference

The table separates three related questions: whether sparsity itself helps, whether hierarchy matters beyond a flat top-k mask, and whether the selected main setting provides a reasonable balance between route commitment and active capacity.

F Objective and Regularization Variants

This section isolates whether the observed routing behavior depends primarily on the selected auxiliary objective or remains stable across objective variants. Table 10 reports ranking quality together with routing-stability indicators for each objective configuration.

Table 10: Objective and regularization variants.

Variant	MRR@20	Neighbor JSD ↓	Route-entropy var. ↓
No auxiliary loss	0.333	0.148	0.094
KNN consistency only	0.336	0.127	0.091
z-loss only	0.337	0.136	0.083
Balance loss only	0.334	0.141	0.090
Consistency + z-loss	0.338	0.124	0.080
Full objective	**0.339**	**0.121**	**0.079**

The two central comparisons are whether the selected objective improves ranking under sparse routing, and whether removing specific auxiliary terms materially changes route sharpness and consistency.

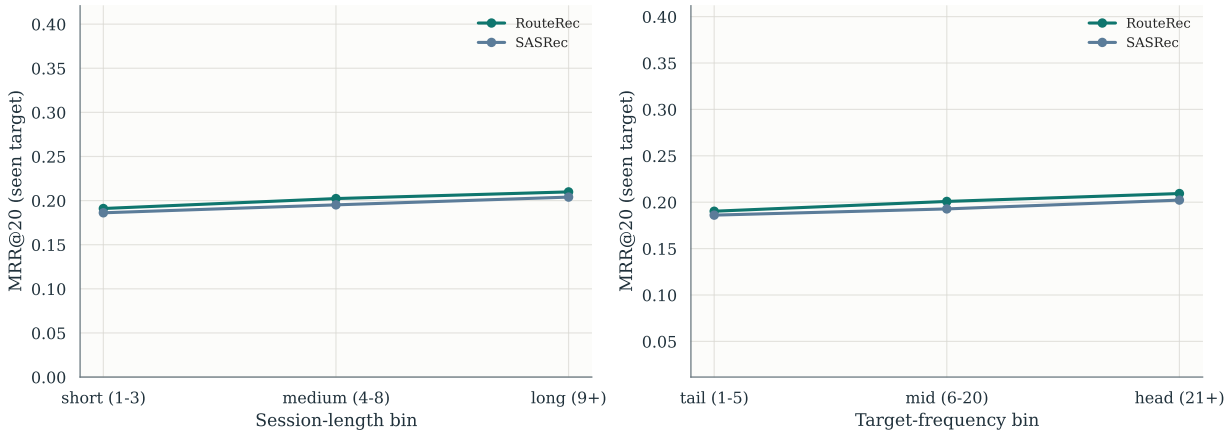
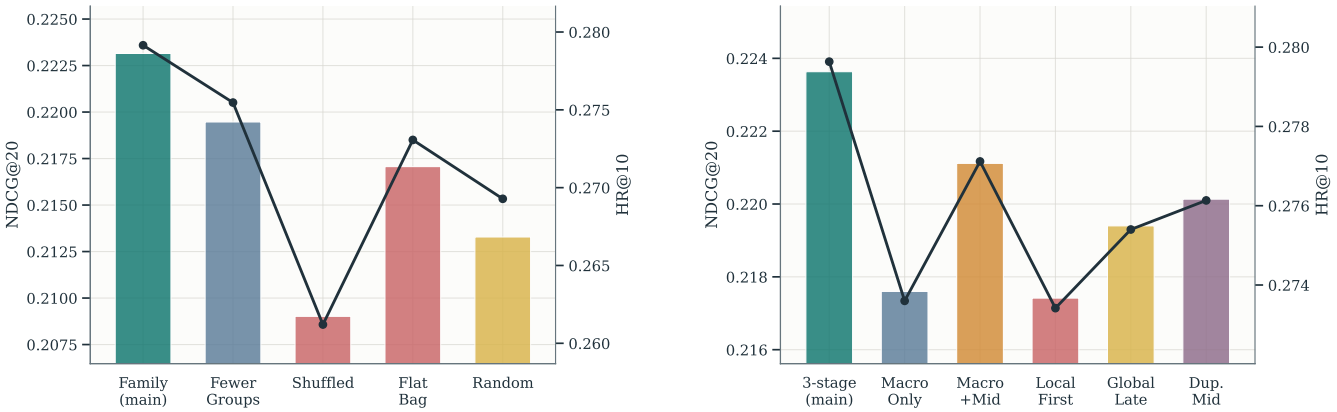


Figure 7: Special-bin analysis reporting where the average RouteRec gain is concentrated across session-length and target-frequency regimes. These bins are coarse difficulty- and exposure-oriented partitions rather than semantically defined behavioral regimes.



(a) Cue-family and expert-group organization variants

(b) Stage-semantics and temporal-layout variants

Figure 8: Extended structural ablations. (a) Cue-family and expert-group organization variants. (b) Stage-semantics, temporal-role controls, and layout variants.

G Full Cost and Efficiency Summary

Table 11 provides a compact cost summary for the claim that the routing branch is lightweight. Rather than turning the appendix into a systems study, this section reports the rough parameter and runtime overhead for representative baselines together with RouteRec's active-parameter view under sparse routing.

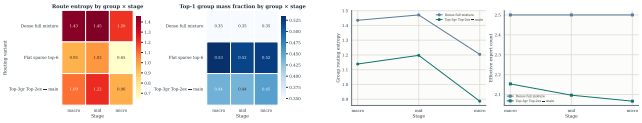
Table 11: Model cost and efficiency summary for all compared methods.

Model	Params	Active params	Train ×	Infer ×	Note
GRU4Rec	1.4	1.4	0.83×	0.88×	Dense recurrent reference
SASRec	1.8	1.8	1.00×	1.00×	Normalization anchor
TiSASRec	2.1	2.1	1.31×	1.22×	Time-aware Transformer
DuoRec	2.0	2.0	1.28×	1.15×	Contrastive sequential
FEARec	2.0	2.0	1.35×	1.24×	Frequency-aware
BSARec	1.9	1.9	1.18×	1.10×	Structural sequential
FAME	2.8	2.8	1.62×	1.48×	Expert-based baseline
DIF-SR	2.4	2.4	1.42×	1.32×	Feature-aware sequential
FDSA	2.3	2.3	1.38×	1.28×	Feature-side attention
RouteRec	3.2	1.6	1.42×	1.18×	Active params: routed subset only

A compact cost comparison is sufficient here to anchor the selective-activation claim and to show that the observed gains are not being attributed solely to an unchecked increase in compute. The timing numbers in this table are measured on a single NVIDIA RTX 3090 24 GB GPU under the same batch-size and evaluation conditions; the ratios serve as a practical reference point rather than as a deployment-grade systems benchmark.

H Extended Routing Diagnostics

This section summarizes the final model's routing structure through aggregate diagnostics. Figure 9 collects expert-group usage, route sharpness, and stage-wise consistency so they can be read together.



(a) Stage-wise family and expert usage heatmaps

(b) Route entropy and effective-expert-count summary

Figure 9: Routing diagnostics. (a) Cue-family and expert usage heatmaps by stage. (b) Route entropy and effective expert count across variants.

These diagnostics are organized from coarse structure to semantic interpretation. Panel (a) shows how cue-family selection and expert usage are distributed by stage, while Panel (b) summarizes route sharpness and effective expert usage across the main sparse-routing variants.

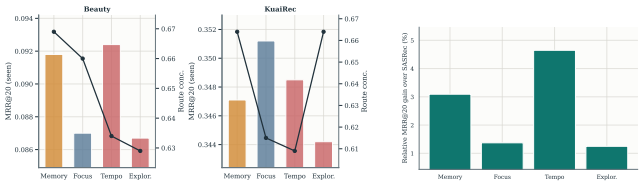
I Extended Behavioral Slice Results

Unlike Appendix C, which analyzes coarse difficulty-oriented bins, this section examines semantically defined behavioral regimes: repeat-heavy, fast exploratory, narrow-focus, and preference-stable slices. Each slice is defined by a simple observable statistic of the interaction history, not by optimized supervision targets.

Table 12: Diagnostic slice definitions used in the behavior-regime analysis.

Slice	Definition rule
Repeat-heavy	Prefix repeat ratio above a fixed percentile threshold
Fast-tempo	Mean inter-event gap below a fixed percentile threshold
Narrow-focus	Category or item-transition entropy below a fixed percentile threshold
Exploration-heavy	Novel-item or low-repeat ratio above a fixed percentile threshold

These slices are used only for post hoc diagnosis. They are computed from simple observable statistics of the interaction history, not from optimized supervision targets, and not from labels learned jointly with the model.



(a) Slice-wise MRR@20 across behavioral regimes

(b) Relative RouteRec gain by slice

Figure 10: Extended behavior-regime analysis. (a) Slice-wise ranking quality across behavioral regimes. (b) Relative RouteRec gain over SASRec by behavioral slice.

Together, the two panels expose dataset-level heterogeneity that would be too detailed for the main paper while making the slice construction explicit enough to avoid circular interpretation.

J Optional Transfer Results

This section reports optional transfer-style results examining whether RouteRec's portable routing interface can be reused across domains with different behavioral and temporal characteristics. The source-target matrix and low-resource curves provide the pairwise and data-efficiency views omitted from the main text.

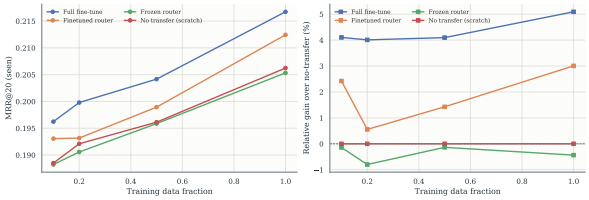


Figure 11: Transfer analysis. Low-resource MRR@20 curves across data fractions comparing full fine-tune, finetuned router, anchor transfer, frozen router, and no-transfer baselines.

Figure 11 shows how the routing interface transfers across data-efficiency regimes. Full fine-tuning and finetuned-router transfer consistently outperform the no-transfer baseline, while the frozen-router setting remains a lightweight alternative at higher data fractions.